

THE VON NEUMANN MODEL

- John W. Mauchly and J. Presper Eckert (1946) principle inventors of the ENIAC, recognized as the first all-electronic, general-purpose digital computer.
- Mauchly and Eckert came up with the idea, of **store program instructions**
- A mathematician named John von Neumann After reading Mauchly and Eckert's proposal for the EDVAC, von Neumann published and publicized the idea.
- All stored-program computers have come to be known as *von Neumann systems* using the *von Neumann architecture*.

THE VON NEUMANN MODEL

Today's version of the stored-program machine architecture satisfies at least the following characteristics :Consists of three hardware systems:

- **A *central processing unit (CPU)***

with a control unit, an *arithmetic logic unit (ALU)*, *registers* (small storage areas), and a program counter; a *main-memory system*, which holds programs that control the computer's operation; and an *I/O system*.

- **Capacity to carry out sequential instruction processing**

- **Contains a single path, either physically or logically,** between the main memory system and the control unit of the CPU, forcing alternation of instruction and execution cycles. This single path is often referred to as the *von Neumann bottleneck*.

THE VON NEUMANN MODEL

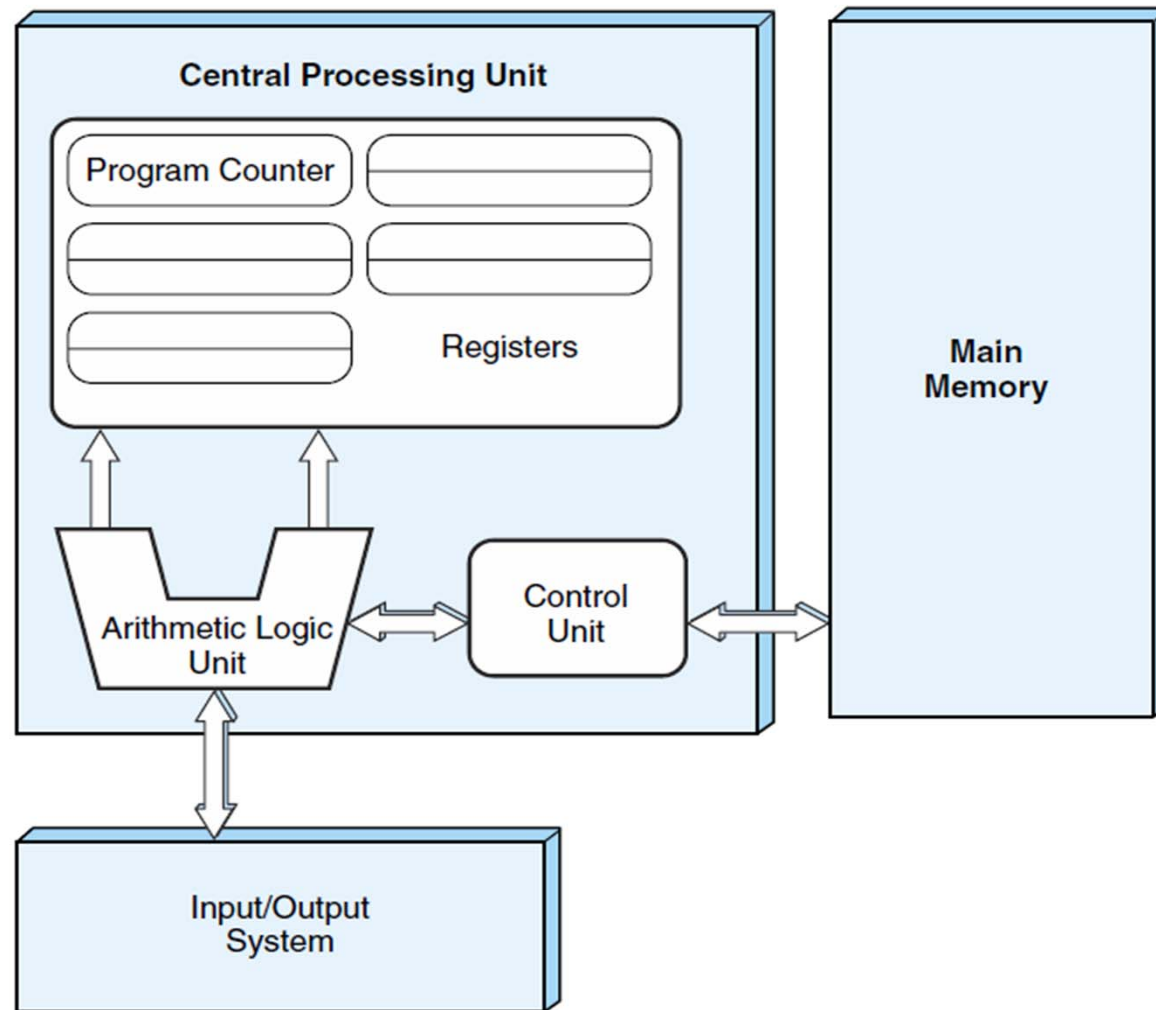


FIGURE 1.4 The von Neumann Architecture

THE VON NEUMANN MODEL

Figure 1.4 shows how these features work together in modern computer systems. Notice that the system shown in the figure passes all of its I/O through the arithmetic logic unit (actually, it passes through the accumulator, which is part of the ALU). This architecture runs programs in what is known as the *von Neumann execution cycle* (also called the *fetch-decode-execute cycle*), which describes how the machine works. One iteration of the cycle is as follows:

1. The control unit fetches the next program instruction from the memory, using the program counter to determine where the instruction is located.
2. The instruction is decoded into a language the ALU can understand.
3. Any data operands required to execute the instruction are fetched from memory and placed into registers within the CPU.
4. The ALU executes the instruction and places the results in registers or memory.

System Bus Architecture

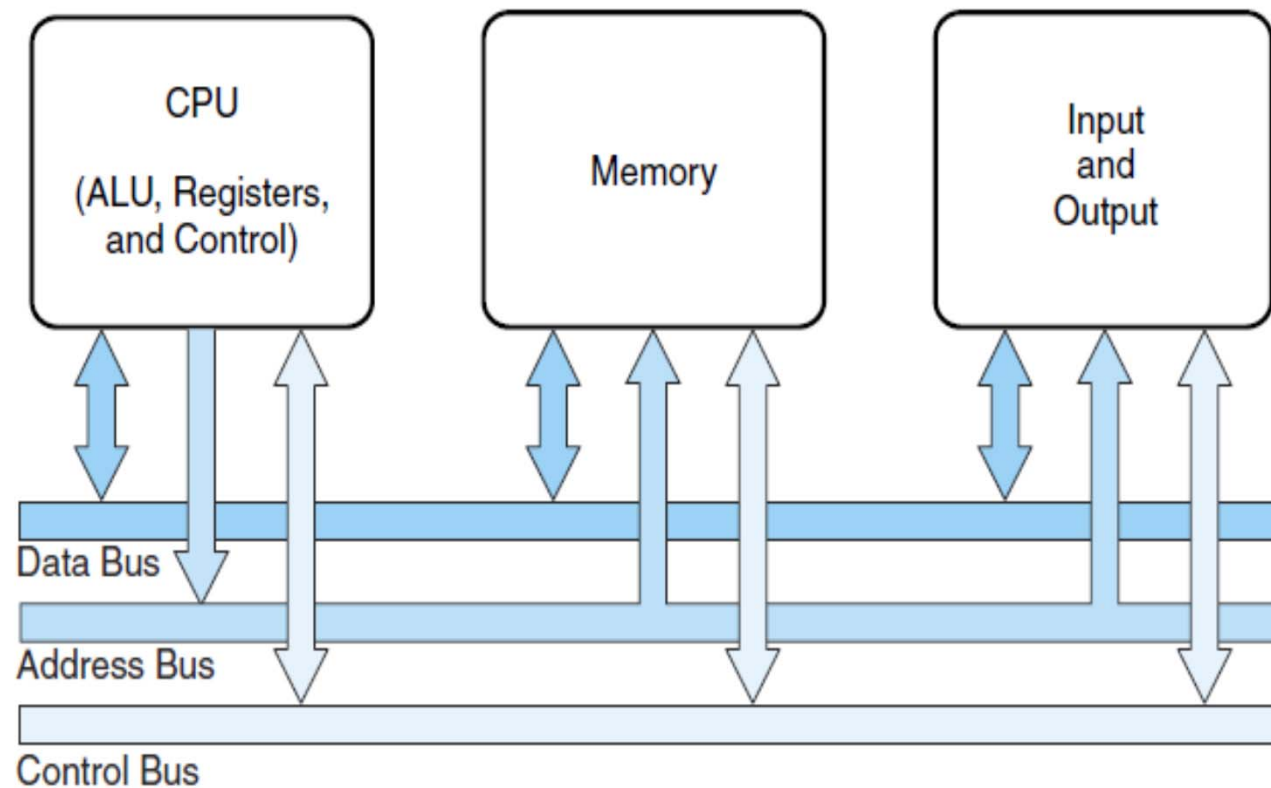


FIGURE 1.5 The Modified von Neumann Architecture, Adding a System Bus

The ideas present in the von Neumann architecture have been extended so that programs and data stored in a slow-to-access storage medium, such as a hard disk, can be copied to a fast-access, volatile storage medium such as RAM prior to execution. This architecture has also been streamlined into what is currently called the *system bus model*, which is shown in Figure 1.5. The data bus moves data from main memory to the CPU registers (and vice versa). The address bus holds the address of the data that the data bus is currently accessing. The control bus carries the necessary control signals that specify how the information transfer is to take place.

Other enhancements to the von Neumann architecture include using index registers for addressing, adding floating point data, using interrupts and asynchronous I/O, adding virtual memory, and adding general registers. You will learn a great deal about these enhancements in the chapters that follow.

Welcome to Assembly Language

- How does assembly language (AL) relate to machine language? One to one
- How do C++ and Java relate to AL? one-to-many
- Is AL portable? no
- Why learn AL?

Assembly Language Applications

- Some representative types of applications:
 - Business application for single platform
 - Hardware device driver
 - Business application for multiple platforms
 - Embedded systems & computer games

(see next panel)

Comparing ASM to High-Level Languages

Type of Application	High-Level Languages	Assembly Language
Business application software, written for single platform, medium to large size.	Formal structures make it easy to organize and maintain large sections of code.	Minimal formal structure, so one must be imposed by programmers who have varying levels of experience. This leads to difficulties maintaining existing code.
Hardware device driver.	Language may not provide for direct hardware access. Even if it does, awkward coding techniques must often be used, resulting in maintenance difficulties.	Hardware access is straightforward and simple. Easy to maintain when programs are short and well documented.
Business application written for multiple platforms (different operating systems).	Usually very portable. The source code can be recompiled on each target operating system with minimal changes.	Must be recoded separately for each platform, often using an assembler with a different syntax. Difficult to maintain.
Embedded systems and computer games requiring direct hardware access.	Produces too much executable code, and may not run efficiently.	Ideal, because the executable code is small and runs quickly.

Source: <http://www.courtesy-language.com/asm-asm-compare.html>

1. How do the assembler and linker work together?
2. How will studying assembly language enhance your understanding of operating systems?
3. What is meant by a *one-to-many relationship* when comparing a high-level language to machine language?
4. Explain the concept of *portability* as it applies to programming languages.
5. Is the assembly language for the Intel 80x86 processor family the same as those for computer systems such as the Vax or Motorola 68x00?
6. Give an example of an *embedded systems* application.
7. What is a device driver?
8. Is type checking on pointer variables stronger in assembly language or in C++?
9. Name two types of applications that would be better suited to assembly language than a high-level language.
10. Why would a high-level language not be an ideal tool for writing a program that directly accesses a particular brand of printer?
11. Why is assembly language not usually used when writing large application programs?
12. *Challenge:* Translate the following C++ expression to assembly language, using the example presented earlier in this chapter as a guide: $X = (Y * 4) + 3$

1. An assembler converts source-code programs from assembly language into machine language. A linker combines individual files created by an assembler into a single executable program.
2. Assembly language is a good tool for learning how application programs communicate with the computer's operating system, via interrupt handlers, system calls, and common memory areas. Assembly language programming also helps when learning how the operating system loads and executes application programs.
3. In a *one-to-many* relationship, a single statement expands into multiple assembly language or machine instructions.
4. A language whose source programs can be compiled and run on a wide variety of computer systems are said to be *portable*.
5. No. Each assembly language is based on either a processor family or specific computer.
6. Some examples of embedded systems applications are automobile fuel and ignition systems, air-conditioning control systems, security systems, flight control systems, hand-held computers, modems, printers, and other intelligent computer peripherals.

7. *Device drivers* are programs that translate general operating system commands into specific references to hardware details that only the manufacturer would know.
8. C++ does not allow a pointer of one type to be assigned to a pointer of another type. Assembly language has no such restriction regarding pointers.
9. Applications suited to assembly language: Hardware device driver, and embedded systems and computer games requiring direct hardware access.
10. A high-level language may not provide for direct hardware access. Even if it does, awkward coding techniques must often be used, resulting in possible maintenance problems.
11. Assembly language has minimal formal structure, so structure must be imposed by programmers who have varying levels of experience. This leads to difficulties maintaining existing code.
12. Code for the expression: $X = (Y * 4) + 3$:

<code>mov</code>	<code>eax, Y</code>	<code>;</code>	<code>move Y to EAX</code>
<code>mov</code>	<code>ebx, 4</code>	<code>;</code>	<code>move 4 to EBX</code>
<code>imul</code>	<code>ebx</code>	<code>;</code>	<code>EAX = EAX * EBX</code>
<code>add</code>	<code>eax, 3</code>	<code>;</code>	<code>add 3 to EAX</code>

Virtual Machine Concept

- Virtual Machines
- Specific Machine Levels

Virtual Machines

- Tanenbaum: [Virtual machine concept](#)
- Programming Language analogy:
 - Each computer has a native machine language (language L0) that runs directly on its hardware
 - A more human-friendly language is usually constructed above machine language, called Language L1
- Programs written in L1 can run two different ways:
 - [Interpretation](#) – L0 program interprets and executes L1 instructions one by one
 - [Translation](#) – L1 program is completely translated into an L0 program, which then runs on the computer hardware

Translating Languages

English: Display the sum of A times B plus C.



C++: `cout << (A * B + C);`



Assembly Language:

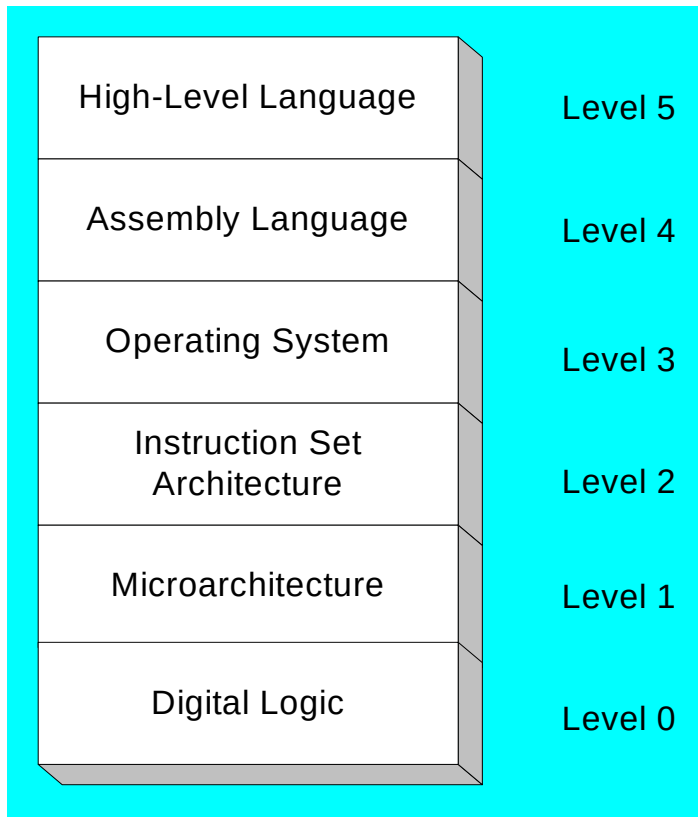
```
mov eax,A
mul B
add eax,C
call WriteInt
```



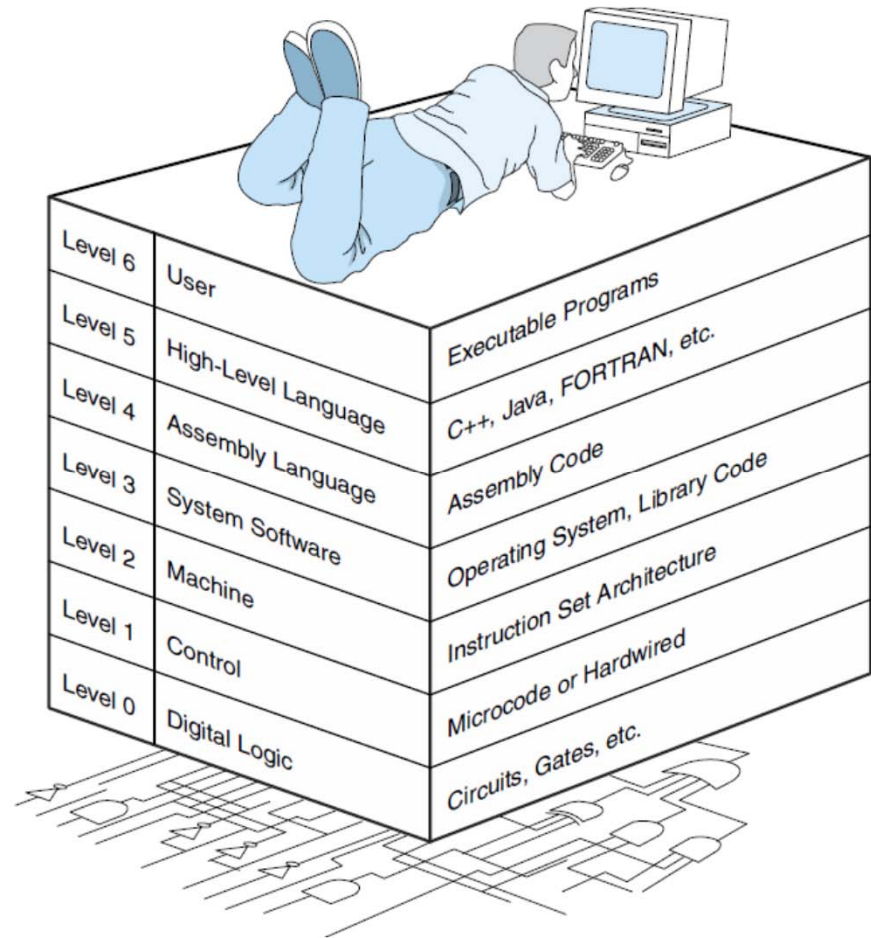
Intel Machine Language:

```
A1 00000000
F7 25 00000004
03 05 00000008
E8 00500000
```

Specific Machine Levels



(descriptions of individual levels follow . . .)



High-Level Language

- Level 5
- Application-oriented languages
 - C++, Java, Pascal, Visual Basic . . .
- Programs compile into assembly language (Level 4)

Assembly Language

- Level 4
- Instruction mnemonics that have a one-to-one correspondence to machine language
- Calls functions written at the operating system level (Level 3)
- Programs are translated into machine language (Level 2)

Operating System

- Level 3
- Provides services to Level 4 programs
- Translated and run at the instruction set architecture level (Level 2)

Instruction Set Architecture

- Level 2
- Also known as **conventional machine language**
- Executed by Level 1 (microarchitecture) program

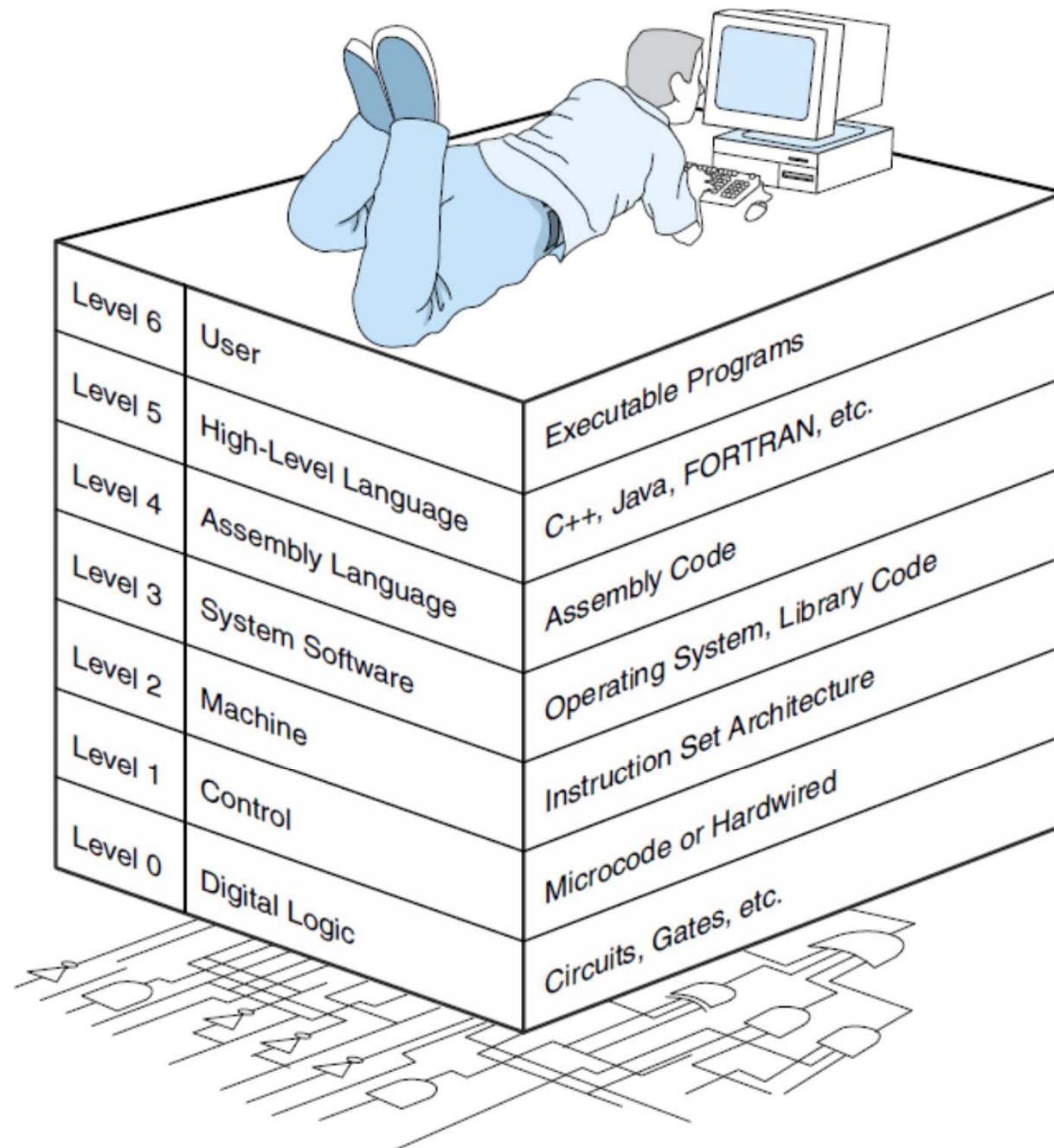
Microarchitecture

- Level 1
- Interprets conventional machine instructions (Level 2)
- Executed by digital hardware (Level 0)

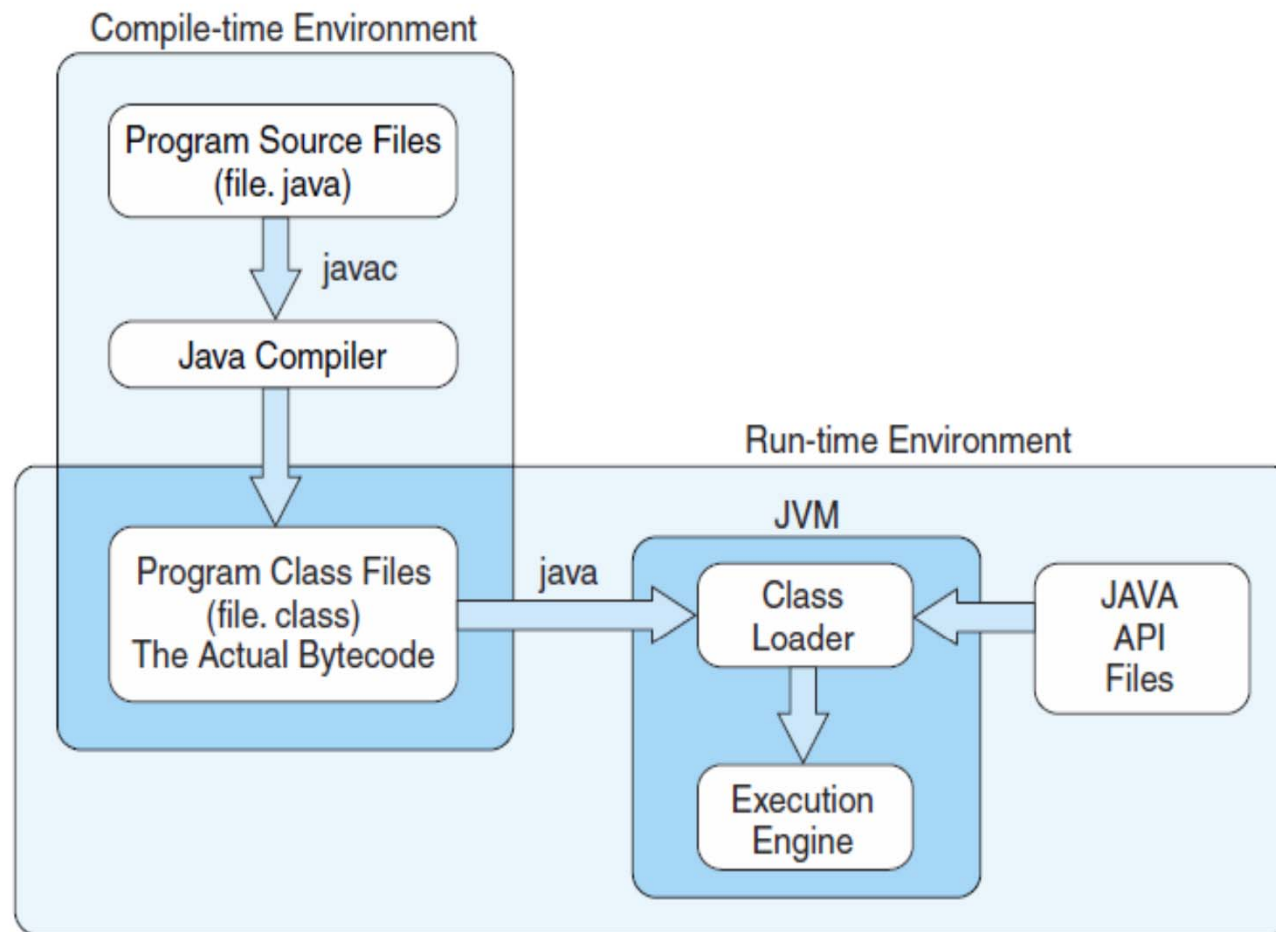
Digital Logic

- Level 0
- CPU, constructed from digital logic gates
- System bus
- Memory
- Implemented using bipolar transistors

next: Data Representation



Java Virtual Machine



1. In your own words, describe the *virtual machine* concept.
2. Why don't programmers use the native language of a computer to write application programs?
3. (*True/False*): When an interpreted program written in language L1 runs, each of its instructions is decoded and executed by a program written in language L0.
4. Explain the technique of translation when dealing with languages at different virtual machine levels.
5. How does the Intel IA-32 processor architecture demonstrate an example of a virtual machine?
6. What software permits compiled Java programs to run on almost any computer?
7. Name the six virtual machine levels named in this section, from lowest to highest.
8. Why don't programmers write applications in microcode?
9. Conventional machine language is used at which level of the virtual machine shown in Figure 1-1?
10. Statements at the assembly language level of a virtual machine are translated into statements at which other level(s)?

1. Computers are constructed in layers, so that each layer represents a translation layer from a higher-level instruction set to a lower level instruction set.
2. It is enormously detailed and consists purely of numbers. Hard for humans to understand.
3. True
4. An entire L1 program is converted into an L0 program, by an L0 program specifically designed for this purpose. Then the resulting L0 program is executed directly on the computer hardware.
5. The IA-32's virtual-86 operating mode emulates the architecture of the Intel 8086/8088 processor, used in the original IBM Personal Computer.
6. Java byte code is a low-level language that is quickly executed at run time by a program known as a Java virtual machine (JVM).
7. Digital logic, microarchitecture, instruction set architecture, operating system, assembly language, high-level language.
8. The specific microarchitecture commands are often a proprietary secret. Also, microcode programming is impractical because it often requires 3 or 4 microinstructions to carry out a single primitive operation.
9. Instruction-set architecture
10. Levels 2 and 3.